Kobo Document Scanner — Code Documentation

Purpose

This repository implements a mobile/web hybrid application that scans identity documents using Microblink (BlinkID), formats parsed data, and sends person-of-record (POR) data to KoboCollect (via a Capacitor plugin). The app supports multiple languages and dark theme.

High-level Architecture

- UI: Angular (v19) with Material components. Main UI logic lives in `AppComponent`.
- Scanning: `BlinkIdScanningService` wraps Microblink BlinkID Capacitor plugin to capture front/back images and parsed fields.
- Native bridge: `src/plugins/send-por-data.plugin.ts` (Capacitor plugin wrapper) sends prepared payloads to Kobo / native code.
- Utilities: small helpers for date parsing, logging, and translations.
- i18n: `src/assets/i18n/*.json` contains translations for UI and formatted results.

Project Structure (important files/folders)

- `src/app/components/app.component.ts` — main UI controller (scan actions, formatting, language & theme toggles).
- `src/app/services/blink-id-scanning.service.ts` — scanner integration and `ScanResult` interface.
- `src/plugins/send-por-data.plugin.ts` — Capacitor plugin interface used to send data.
- `src/app/services/theme.service.ts` — theme toggling & persistence.
- `src/app/services/translation.service.ts` — translation initialization and helpers.
- `src/app/utils/date-util.ts` — `calculateAge()` / `getAge()` used by `AppComponent` (with tests).
- `src/app/utils/log-util.ts` — long message logging helpers.
- `src/assets/i18n/` — `en.json`, `da.json`, `ps.json` for UI/localized keys.
- `src/environments/` — environment configuration for dev/prod (keys like MICROBLINK_LICENSE_KEY).
- `android/` — native Android project; plugin wiring and BlinkID SDK configuration live here.

Key Entities & Responsibilities

- `AppComponent` (`src/app/components/app.component.ts`)
    - Initiates scans: `onScanSingleSideClick()`, `onScanMultiSideClick()`.
    - Handles language selection and persists it (`@capacitor/storage`).
    - Formats scanned `ScanResult[]` via `formatResults()` and displays `scanResults` string.
    - Sends data to Kobo via `sendDataToKoboCollect()` using `SendData.sendData` with a retry wrapper.
    - Uses `calculateAge()` utility for age calculation.
- `ScanResult` (exported from `blink-id-scanning.service.ts`)

    - Shape used throughout app: `frontData`, `backData`, `frontImage`, `backImage`, `dependentsInfo`, `DocumentFace`.
    - Consider tightening types (currently `dependentsInfo: any`) and marking optional fields.

- `BlinkIdScanningService` (`src/app/services/blink-id-scanning.service.ts`)
    - Creates and configures BlinkID recognizers.
    - Provides `scanDocumentMultiSide()` and `scanDocumentSingleSide()` which return `ScanResult[]`.
    - Normalizes raw plugin result fields into `ScanResult` shape.

- `SendData` plugin (`src/plugins/send-por-data.plugin.ts`)
    - Capacitor plugin wrapper with typed `sendData(...)` signature.
    - Expects `dependentsInfo` as string — calling code stringifies non-string values.
    - Ensure native implementation documents accepted payload shape and error cases.

- `ThemeService` (`src/app/services/theme.service.ts`)
    - Manages CSS class toggling on `document.body` and persists `isDarkTheme` in `localStorage`.
    - Suggestion: inject `DOCUMENT` for better testability.

- `TranslationService` (`src/app/services/translation.service.ts`)
    - Wraps `@ngx-translate/core` loader and init logic. Provides `useLanguage()` helper.

Utilities

- `date-util.ts` — parsing multiple DOB formats and returning age.
- `log-util.ts` — chunked console logging for large payloads.

i18n Keys Used

- Results: `results.fullName`, `results.dateOfBirth`, `results.documentNumber`, `results.fathersName`, `results.address`, `results.sex`, `results.dateOfIssue`, `results.documentAdditionalNumber`, `results.dateOfExpiry`.
- Errors/UI: `error.scanFailed`, `error.cardNotSupported`, `close`.

Build & Test

- Build: `npm run build` (calls `ng build`).
- Serve: `npm start` (calls `ng serve`).
- Tests: `npm test` (Karma + Jasmine). Note: test bootstrap is `src/test.ts` and `tsconfig.spec.json` must include `node` types for `require` usage.

Troubleshooting / Known Issues

- Karma may report "Executed 0 of 0" if no spec files are present or file globs are misconfigured. Ensure `src/**/*.spec.ts` exists and `tsconfig.spec.json include` contains `src/test.ts` and spec patterns.

- When installing dev dependencies, `--legacy-peer-deps` might be required to avoid Capacitor peer conflicts (we used this for `@types/node`).

Security & Permissions

- The app requires camera and file permissions on Android. Verify `AndroidManifest.xml` and runtime permission requests.
- Avoid logging sensitive PII in production logs. Use `LogUtil` with environment gating.

Quick Links

- App component: `src/app/components/app.component.ts`
- Scan service: `src/app/services/blink-id-scanning.service.ts`
- Plugin: `src/plugins/send-por-data.plugin.ts`
- Date util: `src/app/utils/date-util.ts`
- Log util: `src/app/utils/log-util.ts`
- i18n folder: `src/assets/i18n`